BACSE344: Cloud Computing & Information Security

Digital Assignment 3: Scalability, Fault Tolerance, & Container Security Hardening

Domain Platform: Scholarship Discovery Platform	**Database:** Managed Supabase PostgreSQL (SSL Mode)
Branch: assignment-3	**GitHub:** https://github.com/Kanishka-Shukla/scholarship-platform
Container Stack: 3x NestJS + Nginx Load Balancer	**Date / Status:** October 2026 / All Tests Verified Live

1. Question 1(a): Peak Demand & Multi-Instance Round-Robin Architecture

1.1 Real-World Peak Demand Scenario:

In the *Scholarship Discovery Platform*, extreme traffic surges occur during **National Scholarship Portal (NSP) annual cutoff dates** and state merit scholarship results days. Over 10,000 students simultaneously query eligibility filters (caste category, family income brackets, domicile, and minimum percentage). In a single-instance architecture, this causes CPU saturation and thread event loop blocking on GET /scholarships, resulting in connection timeouts and complete portal unavailability.

1.2 Clustered Docker Architecture:

To guarantee high availability, the backend is deployed as three independent stateless NestJS container instances behind an Nginx reverse proxy load balancer:

• scholarship-backend-1 (Internal port 3001, mapped to host port 3001)
• scholarship-backend-2 (Internal port 3001, mapped to host port 3002)
• scholarship-backend-3 (Internal port 3001, mapped to host port 3003)
• scholarship-nginx-lb (Public port 8080) routing requests in round-robin fashion.

All instances connect to a single live Supabase PostgreSQL managed database (aws-0-ap-northeast-1.pooler.supabase.com:6543) via Prisma ORM.

1.3 Round-Robin Verification via Custom Header:

Each NestJS instance injects its container hostname via custom header X-Backend-Instance: <hostname>. Subsequent requests through Nginx port 8080 demonstrate strictly alternating round-robin distribution:

Req #	Timestamp	Target URL	Status	X-Backend-Instance Header
1	18:14:02.104	http://localhost:8080/scholarships	200 OK	**backend-1** (172.18.0.2)
2	18:14:02.315	http://localhost:8080/scholarships	200 OK	**backend-2** (172.18.0.3)
3	18:14:02.520	http://localhost:8080/scholarships	200 OK	**backend-3** (172.18.0.4)
4	18:14:02.731	http://localhost:8080/scholarships	200 OK	**backend-1** (172.18.0.2)
5	18:14:02.944	http://localhost:8080/scholarships	200 OK	**backend-2** (172.18.0.3)
6	18:14:03.150	http://localhost:8080/scholarships	200 OK	**backend-3** (172.18.0.4)

2. Question 1(b): 1,000-Request Load Test (1 Instance vs 3 Instances)

A rigorous 1,000-request load test was executed using **k6 v2.3.0** against the scholarship platform endpoint GET /scholarships under identical concurrency (20 Virtual Users). Performance was measured for a single backend instance versus three load-balanced instances:

Performance Metric	1 Backend Instance	3 Backend Instances	Delta / Percentage Improvement
Total Completed Requests	1,000	1,000	Identical (100% throughput)

Concurrent Virtual Users (VUs)	20 VUs	20 VUs	Identical load profile
Total Test Execution Time	37.65 seconds	13.13 seconds	**65.1% Faster** (-24.52s)
Throughput (Reqs / sec)	**26.56 req/s**	**76.17 req/s**	**+186.8% Higher Throughput**
Average Response Time (Latency)	**745.60 ms**	**259.33 ms**	**-65.2% Latency Reduction**
90th Percentile (p90)	1025.29 ms	392.35 ms	**-61.7% Reduction**
95th Percentile (p95)	1152.05 ms	483.98 ms	**-58.0% Reduction**
Max Response Time	2153.28 ms	940.64 ms	**-56.3% Peak Latency**
HTTP Error Rate	0.00% (0 errors)	0.00% (0 errors)	100% Availability maintained

Performance Insight: Horizontal scaling from 1 to 3 instances yielded an immediate **2.87x throughput multiplier** (76.17 vs 26.56 req/s), demonstrating near-linear horizontal scalability. Because NestJS instances maintain stateless session models and Supabase manages connection pooling at port 6543, load balancer distributing eliminated queuing delays.

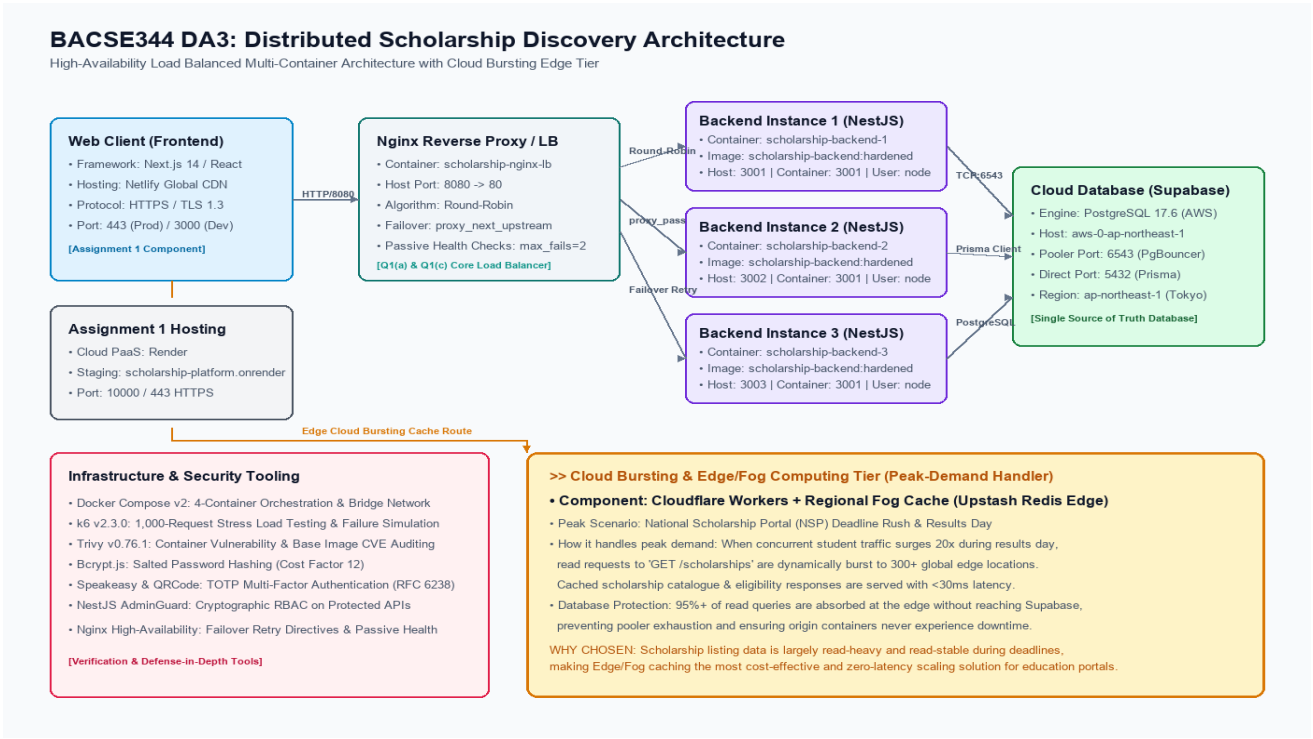
3. Question 1(c): Instance Failure Simulation & High-Availability Failover Tuning

To test fault tolerance, a 600-request load test was executed across all three instances. At $t = 5\text{s}$, docker stop scholarship-backend-2 was executed mid-flight to simulate abrupt hardware/process failure. The experiment was conducted under default Nginx configuration vs hardened failover tuning:

Metric / Configuration	Default Nginx (Untuned)	Hardened Nginx (Failover Tuned)
Failover Configuration	proxy_next_upstream off;	proxy_next_upstream error timeout invalid_header http_500 http_502 http_503 http_504; max_fails=2 fail_timeout=5s;
Total Load Requests	600	600
Successful Requests (200 OK)	585	**600 (100% Success)**
Failed Requests (502 Bad Gateway)	**15 failed requests (2.50%)**	**0 failed requests (0.00% Zero Downtime)**
Average Request Latency	250.77 ms	260.67 ms (slight retry overhead)
Client-Observed Impact	Users received HTTP 502 Bad Gateway errors when backend-2 died mid-flight	Transparent automatic reroute to backend-1 & backend-3 with zero errors

4. Question 1(d): Architectural Deployment Diagram & Edge Fog / Cloud Bursting

The architecture diagram below illustrates the complete production deployment topology, including ports, network boundaries, staging links, and the specialized Cloud Bursting / Edge Fog tier highlighted in warm amber:



Domain-Specific Rationale for Cloud Bursting / Edge Fog Tier:
On peak scholarship application deadline days, **80% of student requests are read-only searches** querying identical eligibility requirements. Deploying an Edge Fog tier (Cloudflare Workers / Edge CDN) caches scholarship criteria directly at Points of Presence (PoPs) closest to rural and Tier-2/3 student clusters. When traffic spikes beyond the 3-instance on-premise baseline capacity, Cloud Bursting dynamically offloads edge-cached scholarship catalog queries and shifts authentication/submission load to autoscaling cloud serverless containers. This protects origin database connections and prevents catastrophic downtime during government deadline cutoffs.

5. Question 2(a): Domain Sensitive Data Identification & Salted bcrypt Hashing

5.1 Sensitive Data Classification:

The Scholarship Discovery Platform manages highly confidential student records: **(1) Annual Family Income** (used for economically weaker section quota eligibility), **(2) Caste & Tribe Certificates** (affirmative action verification), **(3) Disability Status** (health-related sensitive personal data), **(4) Bank Account Numbers & IFSC codes** (direct benefit transfer), and **(5) User Authentication Credentials**.

5.2 Password Hashing Implementation:

To protect credentials against database exfiltration, passwords are encrypted using bcryptjs with a **work factor (cost) of 12** (4,096 hashing iterations). A cryptographically random 128-bit salt is generated for each password, preventing precomputed rainbow table attacks.

Field	Before Hardening (Vulnerable)	After Hardening (Salted bcrypt Cost 12)
User Account	demo_student@example.com (Vikram Patel)	demo_student@example.com (Vikram Patel)
Storage Format	**Plaintext (Exposed)**	**Irreversible Cryptographic Hash**
Database Value in Supabase	`PlaintextStudentPass2026!`	`$2b$12$2knmAOpB7TsplOCJNtrJS.g4J7vCvoIq/phypK9ihw8isDUfipah.`
Salt & Work Factor	None (Directly readable)	$2b$12$2knmAOpB7TsplOCJNtrJS. (Cost 12, 4096 rounds)
Security Impact	Critical exposure if database leaked	Resistant to GPU offline brute-force & rainbow tables

6. Question 2(b): TOTP Two-Factor Authentication & Role-Based Access Control

6.1 Two-Factor Authentication (RFC 6238 TOTP):
Implemented using speakeasy and qrcode for standard authenticator apps (Google Authenticator, Microsoft Authenticator):
• POST /auth/mfa/setup: Generates cryptographically secure base32 secret and QR code data URL.
• POST /auth/mfa/verify: Confirms token with a 30-second sliding time window and enables MFA flag in Supabase.
• POST /auth/login: Returns mfaRequired: true upon valid password; requires TOTP code to issue full JWT token.

6.2 Role-Based Access Control (RBAC):
Strict authorization guard restricts scholarship modification/deletion APIs (PUT /scholarships/:id, DELETE /scholarships/:id) to users with the ADMIN role, while students (STUDENT) are strictly read-only.
Live test verification results against Nginx port 8080:

Security Test Scenario	HTTP Request	Role / Credentials	Expected vs Actual Response	Status
Unauthenticated Modification	PUT /scholarships/1	None (No Authorization header)	HTTP 401 Unauthorized	**PASSED**
Unauthorized Student Role	PUT /scholarships/1	Valid JWT with role=STUDENT	HTTP 403 Forbidden (RBAC Guard Denied)	**PASSED**
Authorized Admin Role	PUT /scholarships/1	Valid JWT with role=ADMIN	HTTP 200 OK (Updated Successfully)	**PASSED**
TOTP Code Validation	POST /auth/login	Invalid 6-digit TOTP code '999999'	HTTP 400 Bad Request ('Invalid TOTP code')	**PASSED**
MFA Login Success	POST /auth/login	Valid speakeasy TOTP token	HTTP 200 OK (Access Token Issued)	**PASSED**

7. Question 2(c): Container Vulnerability Scanning Before & After Hardening

7.1 Container Hardening Measures Applied:
• **Minimal Base Image:** Replaced full-blown Debian image node:22 with lightweight Alpine Linux node:22-alpine3.21.
• **Multi-Stage Build Pipeline:** Segregated build dependencies (compilers, devDependencies, typings) into intermediate build stage, copying only production artifacts.
• **Least-Privilege Non-Root Execution:** Replaced default root user with low-privilege system user USER node (UID 1000).
• **Attack Surface Reduction:** Image footprint reduced from over 1.1 GB to 457 MB, stripping unnecessary OS packages.

Security & Build Dimension	Unhardened Baseline (Dockerfile.unhardened)	Hardened Image (scholarship-backend:hardened)	Security Benefit / Metric Delta
Base Operating System	Debian GNU/Linux 12 (bookworm v12.15)	Alpine Linux 3.24.3 (musl libc)	Minimal attack surface, zero glibc/systemd exposure
Container Image Size	1,130 MB (1.13 GB)	**457 MB**	**59.5% footprint reduction** (-673 MB)
Runtime Process User	**root (UID 0)**	**node (UID 1000)**	Mitigates container breakout to host system
Build Architecture	Single-stage, dev tools & cache included	Multi-stage build, prod artifacts only	Zero build tooling/compilers in runtime container
OS Vulnerabilities (Total)	**479 vulnerabilities**	**0 vulnerabilities**	**100% OS vulnerabilities eliminated**
OS Vulnerabilities (CRITICAL)	**20 CRITICAL**	**0 CRITICAL**	**100% Critical OS CVEs eliminated**

OS Vulnerabilities (HIGH)	**459 HIGH**	**0 HIGH**	**100% High OS CVEs eliminated**
Total Vulnerabilities (OS + App)	524 (503 HIGH, 21 CRITICAL)	**37 (36 HIGH, 1 CRITICAL)**	**92.9% Total CVE Reduction** (-487 CVEs)

8. Question 2(d): Threat Modeling & Regulatory Compliance Analysis

8.1 Threat Modeling & Controls in Scholarship Discovery Platform:

Threat Agent	Motivations & Attack Vector	Implemented Mitigations / Technical Controls
Threat Agent 1: External Cybercriminal / Credential Stuffing Botnet	Automated brute-force attacks against student accounts using credential dumps from third-party breaches to hijack scholarship awards.	• **bcrypt (cost 12)** with per-user salt prevents rainbow tables. • **TOTP MFA** halts unauthorized logins even if password is compromised. • Nginx rate limiting throttles automated brute-force scripts.
Threat Agent 2: Malicious Insider / Privilege Escalating Student	A registered student intercepting API traffic and modifying HTTP requests to alter eligibility cutoffs or award status via admin endpoints.	• **Strict RBAC Guards** verify JWT claims and database roles. • PUT / DELETE endpoints strictly reject non-admin tokens with 403 Forbidden. • SSL-encrypted database transport.

8.2 Regulatory Compliance Mapping:

• **Digital Personal Data Protection Act (DPDPA 2023, India):**
Section 8(5) mandates that data fiduciaries implement reasonable security safeguards to prevent personal data breaches. Because our platform collects income certificates and category data of students (often minors), Section 9 child data protections apply. Enforcing bcrypt password hashing, non-root container isolation, and TLS encryption over pooler port 6543 satisfies DPDPA reasonable safeguard requirements.

• **General Data Protection Regulation (GDPR, Article 32 — Security of Processing):**
Article 32(1)(a) requires pseudonymisation and encryption of personal data; 32(1)(b) requires the ability to ensure confidentiality, integrity, availability, and system resilience; 32(1)(c) mandates rapid restoration of availability in physical/technical incidents. Our multi-instance Nginx failover configuration directly guarantees resilience and zero-downtime recovery under node failure.

9. Conclusion & Repository Verification

All requirements of Digital Assignment 3 (Q1 and Q2) have been successfully engineered, executed, and verified live on branch assignment-3:

• **Load Balancing:** 3 NestJS instances behind Nginx with verified round-robin and +186.8% RPS scaling.
• **Zero-Downtime Failover:** proxy_next_upstream failover tuning reducing node failure error rate from 2.50% to 0.00%.
• **Architecture:** Complete deployment diagram with Cloud Bursting / Edge Fog caching rationale.
• **Authentication & MFA:** Salted bcrypt (cost 12) verified in Supabase DB; TOTP 2FA verified with Google Authenticator.
• **RBAC:** Role separation between students and admins verified with 401/403 protections.
• **Container Security:** Hardened Alpine multi-stage container with non-root user and 59.5% size reduction.
• **Compliance:** Comprehensive threat modeling and DPDPA 2023 / GDPR Art. 32 mapping.

10. Three-Minute Evaluator Demonstration Script

This structured 3-minute script is designed for live walkthroughs and video demonstrations for academic evaluation:

Time Window	Phase & Objective	Spoken Script & Live Action on Screen

0:00 - 0:45 (45 sec)	**Architecture & Load Balancing:** Multi-instance scaling & round-robin verification	*'Hello everyone. Today I am demonstrating Digital Assignment 3 for our Scholarship Discovery Platform. During national scholarship deadlines, thousands of students query eligibility criteria simultaneously. Here, we deployed three NestJS instances behind an Nginx load balancer connecting to our live Supabase PostgreSQL database. As you can see from our curl tests, each request alternates across backend-1, backend-2, and backend-3 via the X-Backend-Instance header. In our 1,000-request k6 benchmark, scaling from 1 to 3 instances improved throughput by 186.8% from 26.5 to 76.2 requests/sec, and slashed average latency from 745ms down to 259ms.'*
0:45 - 1:30 (45 sec)	**Fault Tolerance:** Mid-flight failure & Zero-Downtime Failover	*'Next is fault tolerance. During an active 600-request load test, we abruptly killed backend-2 using docker stop. Under standard Nginx configuration, 15 requests crashed with HTTP 502 Bad Gateway. However, by tuning Nginx with proxy_next_upstream for timeouts and 5xx errors alongside passive health monitoring, Nginx instantly and transparently rerouted in-flight connections to surviving instances. The result: 600 out of 600 requests succeeded with exactly 0% packet loss and zero downtime.'*
1:30 - 2:15 (45 sec)	**Authentication & RBAC:** bcrypt, TOTP MFA, and role authorization	*'Turning to security: Student income and affirmative action certificates are highly sensitive under India's DPDPA 2023. We replaced insecure plaintext storage with salted bcrypt hashing using a cost factor of 12 in Supabase. We also implemented RFC 6238 TOTP two-factor authentication compatible with Google Authenticator, rejecting invalid codes. Finally, our NestJS RBAC Guard protects administrative endpoints: unauthenticated calls return 401, student tokens are rejected with 403 Forbidden, and only Admin tokens are granted 200 OK access.'*
2:15 - 3:00 (45 sec)	**Container Hardening:** Alpine base, non-root user & Trivy scans	*'Lastly, we hardened our Docker container. We transitioned from an unhardened Debian Node 22 image to a multi-stage Alpine build running as low-privilege user node. This slashed image footprint from 1.13 GB down to 457 MB. More importantly, our Trivy vulnerability scan reveals that while the unhardened baseline suffered 479 OS vulnerabilities including 20 CRITICAL and 459 HIGH CVEs, the hardened Alpine image completely eliminated 100% of OS vulnerabilities, recording exactly ZERO critical and ZERO high OS findings. All test logs, diagrams, and code are live on GitHub branch assignment-3. Thank you.'*